APEX Account Planner: Backend & Reasoning Engine Architectural Post-Mortem

This document tracks the comprehensive chronological debugging journey, architectural shifts, and structural improvements made to the APEX Account Planner backend. It is designed to help junior developers understand:

1. What bugs occurred and why.
2. How the system architecture evolved.
3. How we implemented self-healing logic to handle local LLM anomalies.

---

1. Chronological Timeline of the Debugging Session

Phase 1: Unreachable LLM & Network Blocker

- **Symptom**: The backend logged `Error or timeout before response: timed out` at all reasoning and response layers. The system fell back to a default low-confidence (30%) message: *"AM Response Generation failed/timed out: timed out"*.
- **Diagnosis**:
  - The local backend (port `8002`) tried to connect to a remote LLM server hosting Llama 3.2 at `192.168.148.105:8000`.
  - Standard system proxy settings rerouted local IP traffic, making the host unreachable even though a manual ping worked.
- **Resolution**: Configured the backend shell environment with `set NO_PROXY=192.168.148.105` to bypass the proxy, restoring successful HTTP sockets to Ollama.

Phase 2: CPU-Only Parallel Crash & Latency Collapses

- **Symptom**: After the network connection was established, queries took minutes or timed out.
- **Diagnosis**: The LLM server ran Llama 3.2 on an integrated Intel Arc card operating in CPU-only or low-acceleration mode. Running four reasoning threads concurrently in parallel overwhelmed the CPU cores/RAM (16GB), causing Ollama to drop requests or take several minutes per token.
- **Resolution**: Shifted from a parallel execution model to a sequential thread execution model for local execution (see Section 2).

Phase 3: JSON Format Anomalies & Type Violations

- **Symptom**: The Data thread failed to parse, causing fallback values.
- **Diagnosis**:
  - The LLM output raw percentage signs (e.g. `6.2%` instead of `0.062`) and unquoted placeholders (`N/A`) in numeric fields.
  - The LLM returned lists of strings in the Layer 3 response under `"talking_points"`: `["New features", "Value proposition"]` while Pydantic expected a strict `List[Dict[str, str]]` in Layer 2's structures, causing type violations.
- **Resolution**: Updated data thread prompt parameters to forbid unquoted percent signs and `N/A` text, and isolated the parsing schemas (see Section 3).

Phase 4: Suffix Truncations & Regex Cleanup Failures

- **Symptom**: The AM Response fell back to a default playbook with empty arrays `[]` in the UI.
- **Diagnosis**: The response was truncated due to token limits, ending without a closing double quote and closing curly brace:
  `"why_this_matters": "Data quality issues may impact account plan effectiveness`
  The old parser blindly sliced text up to the last closing brace (the end of the preceding `risks_and_mitigations` list), discarding the truncated property. Since the remaining JSON was technically valid, it parsed successfully but raised validation errors due to missing required fields.
- **Resolution**: Upgraded prefix/suffix slicing logic and implemented unclosed quote healing (see Section 5).

---

2. ARCHITECTURE SECTION: Sequential vs. Parallel Thread Execution

Parallel Thread Model (Cloud Deployment)

- **Design**: Uses a `ThreadPoolExecutor` to execute the 4 reasoning threads (AM, VP, SVP, Data) simultaneously.
- **Problem**: Under local CPU/RAM constraints, concurrent text generation slows down to 1–2 tokens per second per thread. Pre-fill processing bottlenecks on the CPU, causing connection timeouts.

Sequential Thread Model (Local Deployment)

- **Design**: We removed the local data-only bypass (`DATA_ONLY_LOCAL`) that skipped reasoning. Instead, we execute the 4 threads sequentially when `local=True` :

```
if local:
    # Sequential execution prevents CPU/RAM crashes on local Ollama
    am_res = self._run_am_thread(account_data, seq_timeout)
    vp_res = self._run_vp_thread(account_data, market_context, seq_t
    ...
```

- **Pros**: Restricts active memory footprints to a single active LLM context. Ollama never crashes from context overloading.
- **Cons**: Total latency is the sum of all 4 threads (~2.5 to 3.5 minutes total execution). We mitigated this by implementing real-time progress stream logs.

---

3. LAYERS SECTION: OLD vs. NEW Pipeline Comparison

The APEX Account Planner operates in a strict multi-layer structure. Below is the breakdown of how these layers evolved:

Pipeline Layer	OLD Design & Issues	NEW Design & Improvements	Why We Changed It
Layer 1: Intent Classifier	The rule-based entity extractor mapped capitalized words blindly. A query like *"Who are key decision makers..."* mapped `"Who"` as the target account name.	Expanded the entity noise filter list to block question pronouns and verbs: `['Who', 'What', 'Where', 'When', 'Why', 'How', 'Which', 'Is', 'Are', 'They', 'Them']` .	Preventing extraction of question words stops the database retrieval from failing and falling back to a global search.
Layer 2: Data Retrieval	If a targeted account name was not found (e.g. `"Who"`), it defaulted to returning the entire Excel dataset (**119 opportunities**), which flooded LLM prompts.	Implemented `summarize_large_dataset()` . If opportunities exceed `5` , the system calculates aggregates and retains only the top 5 largest deals by value.	Context windows are kept under 500 tokens, eliminating CPU pre-fill overhead and thread timeouts.
Layer 2: Reasoning Threads	Parsers for AM, VP, SVP, and Data threads caught JSON exceptions internally and returned the `'am'` fallback dict. This caused schema validation failures on consensus.	Isolated consensus parsing (`is_consensus=True`). If consensus parsing fails, it raises an exception, allowing the caller to use the proper `ConsensusOutput` fallback.	Separates thread fallbacks from consensus fallbacks, eliminating Pydantic schema validation failures.

Pipeline Layer	OLD Design & Issues	NEW Design & Improvements	Why We Changed It
Layer 3: Response Generator	Truncated outputs were sliced at the last closing brace `}`, discarding trailing properties and causing required fields to be missing.	Implemented balanced-brace slicing. Suffixes are only stripped if the JSON is structurally complete, forcing truncated strings to fail parsing and go to repair.	Prevents silent data loss of truncated fields, forcing the repair engine to restore the properties.

---

4. TIMEOUT & CONSTRAINTS SECTION

Slow CPU execution requires a balance between waiting time and output limits. We adjusted these constraints to fit local hardware:

1. **Wait Time (HTTP Timeouts)**:
    - Raised reasoning thread timeout from **15 seconds** to **180 seconds** (3 minutes).
    - Raised final response timeout from **30 seconds** to **180 seconds** (3 minutes).
    - *Why*: Allows CPU pre-fill and generation loops to finish naturally without premature connection drops.
2. **Capping Generation Length (`num_predict`)**:
    - Reasoning threads (Layer 2) are capped at `"num_predict": 400` tokens (originally unlimited or `256`).
    - Response generation (Layer 3) is capped at `"num_predict": 512` tokens (originally unlimited or `350`).
    - *Why*: Prevents models from entering long-winded hallucination loops, keeping execution times under 3 minutes.
3. **Prompt Engineering Instructions**:
    - Appended strict prompt directives to enforce conciseness: `"\nCRITICAL: Keep output extremely concise. Total JSON output must be under 200 tokens."`

---

5. JSON FORMATTING & SELF-HEALING PARSER SECTION

Small local LLMs often output invalid JSON syntax. The backend uses a unified `repair_json_string` function to automatically heal strings before parsing:

```
[Raw LLM Text]
       |
       ▼
[Standard JSON Parser (json.loads)] ──(Success)──> [Return Dictionary]
       |
    (Fail)
       ▼
[Regex-Based JSON Repairer]
  - Convert single quotes to double quotes ('key': 'val' -> "key": "val")
  - Replace unquoted Python literals (None -> null, True -> true)
  - Escape raw newlines/tabs inside double quotes
  - Count unescaped quotes; if odd, append closing quote "
  - Balance braces and brackets by appending missing } or ]
       |
       ▼
[Standard JSON Parser (json.loads)] ──(Success)──> [Return Dictionary]
       |
    (Fail)
       ▼
[Regex Structural Fallback Extractor] ──────────> [Return partial dict + defaults]
  - Regex-scans for fields and returns a dictionary with default values.
```

Key Repair Regex Implementations

- **Fixing Single Quotes**:
  `repaired = re.sub(r"'\s*(\w+)\s*'\s*:", r'"\1":', repaired)`
- **Healing Truncated Quotes**:

  ```
  unescaped_quotes = len(re.findall(r'(?<!\\)"', repaired))
  if unescaped_quotes % 2 != 0:
      repaired += '"'
  ```

- **Injecting Commas**:
  `repaired = re.sub(r'}\s*{', '}, {', repaired)` (inserted missing comma between objects in lists).

---

6. LLM SECTION

Llama 3.2 (3B Parameters) has unique constraints when run locally:

1. **Escape Characters**: It frequently generates invalid `\'` escapes for apostrophes (e.g. `Accenture\'s`). The repair parser replaces `\'` with `'`.
2. **Numeric Fields**: It tends to output strings or percentage symbols for float fields. Strict formatting rules added to the prompt ensure it returns valid JSON floats:
    - *"Every numeric value must be a raw float or integer, NOT containing a '%' sign."*
    - *"Do NOT output 'N/A' or 'null' for numeric fields."*

---

7. Workspace & Test Restructuring

All backend test scripts have been consolidated into the dedicated `tests` directory under backend/tests to separate tests from core logic:

- `test_agent_orchestrator.py`
- `test_chat.py`
- `test_intent.py`
- `test_intent_classifier.py`
- `test_reasoning_engine.py`
- `test_repair_json.py`
- `test_response_generator.py`

Junior developers: please place any new test files inside this directory.